

Project Documentation (G13)

Chang Hyeji

e12535756@student.tuwien.ac.at

12535756

Eickmeyer Michael

michael.eickmeyer@tuwien.ac.at

11905167

November 23, 2025

1 Overview

For this submission, we have developed the base of our Vulkan renderer in which we will code our tasks in. Our renderer consists of multiple modules, split up as follows:

include/native — Our header files. More precisely, **/core** contains the renderer core, **managers** essential functionality, and **/util** utility functions and classes.

resources — Where shaders, objects, and scene descriptions will be stored. Our renderer automatically parses GLSL and translates it into SPIR-V for rendering.

src — Renderer implementation. The folder structure is analogous to the structure in **include/native**.

Unfortunately, the code is currently unable to display a functioning scene due to a bug in the rendering loop.

2 Planned Steps

- Implement object loading and texturing via *tinyobjloader*, as described in the project proposal
- Fix remaining render base issues
- Implement tasks as per proposal

3 Changes / Additions

Not a lot has changed compared to our project proposal. We still use C++20 in combination with Vulkan, leveraging the AutoVK layer. We expanded the use of third-party libraries to *loguru* for logging and *rapidyaml* for reading *.yaml* files (configs, scene descriptions in the future).